ROS 2-based automatic modular design framework for the TurtleBot 4

Ben Kroener

September 2025

Abstract

The design of control software for robot swarms is challenging, because control is implemented at the individual robot level, while tasks are defined for the entire swarm. To tackle this offline design methods are used, where an optimization problem is created for the complete swarm and solved by running multiple simulations and evaluating the performance. Although this approach achieves good results in simulation, performance often drops when deployed on real robots. This phenomenon is known as the reality gap. Automatic modular design (AutoMoDe) methods were introduced to reduce this performance drop, by using predefined modules for low level conditions and behaviors. So far, these methods have been limited to a small number of simulators and robot platforms. This work therefore introduces a new implementation which extends AutoMoDe to the Gazebo simulator and the TurtleBot 4 platform. It is also designed to be extendable to further robot platforms and simulators in the future. Therefore it provides a foundation that may help address the limitation on hardware and software, to which AutoMoDe methods are deployed to.

Contents

1 Introduction **3**

2 Implementation **3**
2.1 General structure in ROS 2 4
2.2 Controller node 5
2.3 Finite State Machine 6
2.4 Reference Model 6
2.5 Behavior node, Condition node 6
2.6 Behavior and Condition Modules 7
2.7 Interfaces 8
2.8 Development and Testing Tools 9
2.9 Launch files 10

3 How to do **11**
3.1 Install 11
3.1.1 OS installation 11
3.1.2 ROS 2 installation 11
3.1.3 Dev tools and simulator installation 11
3.1.4 Create ROS 2 workspace 12
3.1.5 Create3 installation 12
3.1.6 TurtleBot 4 installation 13
3.1.7 AutoMoDe installation 14
3.1.8 Optional: Add sources to `.bashrc` 14
3.2 Start the example missions 15
3.2.1 Start the robots tb1 and tb2 15
3.2.2 Mission 1: Foraging 16
3.2.3 Mission 2: Find and catch 17
3.2.4 Mission Conclusion 17
3.3 Adding new sensor data 18
3.4 Adding new modules 19
3.5 Add new worlds 21
3.6 AutoMoDe Editor Dr. Kuckling 21

4 Discussion **22**

1 Introduction

The first paper about AutoMoDe was published by Francesca et al. [2014]. An automatic modular design framework was presented, which introduces a concept for training robot swarms with predefined modules. The idea is to reduce the negative impact on performance (effectiveness in task completion), when in simulation trained control software is transformed to real robots. This performance drop is known as reality-gap and is still a major problem in offline-design methods. Since then especially the Demiurge project [1] brought a lot of publications and scientific progress in various directions of the field. A good overview is given by Birattari et al. [2021]. Most of these publications use ARGoS3 [Pinciroli et al., 2012] as simulator and the e-puck [Mondada et al., 2009] as robot platform. Few use also Webots [Michel, 2004] and Mercator [Kegeleirs et al., 2022]. These combinations have effective for research, but the reliance on the same setups has so far limited exploration with other simulators and robot platforms. To broaden the applicability of AutoMoDe, I present an implementation for the Gazebo simulator [Koenig and Howard, 2004] and the robot platform TurtleBot 4 [2]. Using ROS 2 [Macenski et al., 2022] as a middleware, I also enable further adaptations to other platforms and simulators, as it provides a standardized communication interface that is widely supported across different robotic platforms and simulation environments. This work is intended to support further exploration of AutoMoDe and to demonstrate its use on the TurtleBot 4 platform in the Gazebo simulation environment.

2 Implementation

The goal was to create a implementation of the AutoMoDe software, which enables easy adaption to new robot platforms as well as changes in the implementation itself. Such changes include the switch to other modules for the conditions and behaviors as well as changes in the sensor information used. To do this, the implementation is done in Python programming language and uses ROS 2 [Macenski et al., 2022] as SDK, making it widely usable, easy to read and fast to build. ROS 2 also provides inherently good modularity and extensibility, through its architecture based on notes communicate via topics, services and actions. Each node encapsulates a clear functionality which enables separation of concerns. The implementation is adapted to the robot platform TurtleBot 4, with modules matching previous implementations. As a simulator I used Gazebo [3] and the TurtleBot 4 package. This package provides a model of the TurtleBot 4 and ROS 2 support. Following this, I describe the implementation for the TurtleBot 4 as well as explanations for design choices and guidance on further adaptions.

[1] Demiurge project: https://demiurge.be/
[2] TurtleBot 4 webpage: https://clearpathrobotics.com/turtlebot-4/
[3] Gazebo website: https://gazebosim.org/home

2.1 General structure in ROS 2

The implementation consists of five packages: `automode_controller`, `automode_interface`, `automode_tools` , `automode_tb4_sim` and `basic_modules`.
`automode_controller`, is the central package which consists of the controller nodes and a launch file for them, the other ones support it.
`automode_interface` defines the messages and actions used in the controller.
`automode_tools` has testing nodes and utilities for development.
`automode_tb4_sim` gives the robot in the simulation additional needed sensors and provides launch files for the simulation.
`basic_modules` gives an example for the module package, which consists of the behaviors and conditions.
All of them are together in a colcon workspace to ease the workflow of change, build and execute. The focus is on extensibility and modularity.
In Table 1 I give an overview of the ROS 2 entry points, modules and interfaces of the implementation. As the `automode_controller` package together with the `basic_modules` package, is responsible for controlling the robots, I want to point out the structure at runtime and in the implementation. The individual parts are described in detail in the following sections. The Figure 1 shows structure at runtime, visualized by the tool `rqt_graph` from ROS 2. This gives ROS 2 users a known overview over the parts and communication paths. Figure 2 gives a more structural overview of the running system and its implementation. It therefore works as a complement to the `rqt_graph`. Nodes are shown as Squares with ovals as additional files and topics as squares with double lined sides. The focus is on the communication between the nodes as well as the additional files and their affiliation.
The complete implementation supports extensive logs and error handling to speed-up debugging of new parts. All necessary loops in the system work at 20Hz as it is a good compromise between fast reaction and performance.

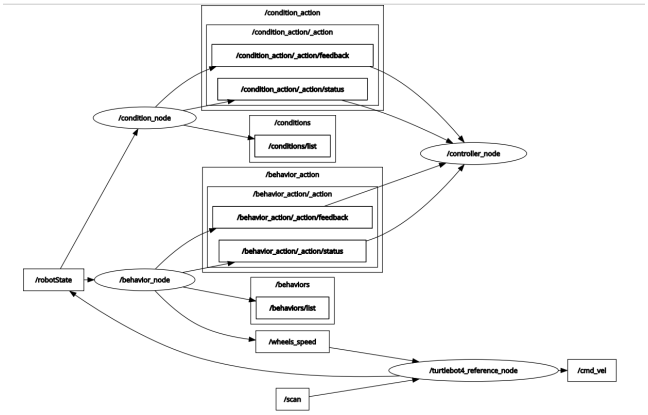

Figure 1: AutoMoDe implementation in rqt_graph while running

Package	Nodes (Entry Points)	Modules / Interfaces
automode_controller	behavior_node condition_node controller_node ref_model_turtlebot4_gz automode_tb4.launch	behavior_node.py condition_node.py controller_node.py ref_model_turtlebot4_gz.py automode_tb4.launch.py
automode_tools	behavior_test_node condition_test_node robotState_test_pub categories_creator	behavior_test_node.py condition_test_node.py robotState_test_pub.py categories_creator.py
basic_modules	(no nodes/entry points)	behaviors/ (dir) conditions/ (dir)
automode_interfaces	(no nodes/entry points)	action/Behavior.action action/Condition.action msg/RobotState.msg
automode_tb4_sim	turtlebot4_simulation.launch add_turtlebot4_simulation.launch tf_broadcaster_launch	turtlebot4_simulation.launch.py add_turtlebot4_simulation.launch.py tf_broadcaster_launch.py ground_sensor_node.py ir_sensor_node.py light_sensor_node.py neighbour_sensor_node.py

Table 1: Packages, nodes, and module/interface files

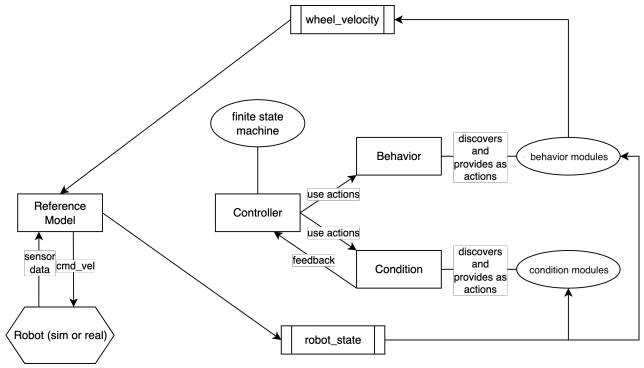

Figure 2: An overview of the AutoMoDe implementation

2.2 Controller node

The controller node is the center of the implementation. It works as a connection between the finite state machine and the behavior as well as the condition actions. For initialization it requests the behavior and condition description and passes them with the `fsm_config` to the finite state machine. If the state or the condition switches, it cancels the active actions and starts the necessary

new ones. The controller has no access to the `robotState` topic to focus only on the high level decision and execution process. It has no information about the current state of the robot but only on the executed actions and state of the finite state machine. This is done for clean separation of concerns and should not be changed. Although implemented in a separate file, the finite state machine runs within the controller node. That enables easy communication. They are connected closely anyway and changing to another decision strategies would need changes in both parts. In addition to that it reduces a lot of hard to read code lines and ROS 2 communication overhead. Especially on startup the controller node logs what finite state machine is created as well as errors if happening. If the finite state machine configuration is new and not working correctly, checking these logs can be helpful.

2.3 Finite State Machine

The finite state machine is at runtime a part of the controller node. It is the decision process of choice for this implementation. With that, it follows older implementation. The finite state machine is actually only a very small data class, but what makes the file very big (400 lines of code), is the parsing of the `fsm_config` with extensive error handling and error output. This output is given via the controller node logs. Behavior modules are represented as states and conditions as edges; only one state but multiple conditions can be active simultaneously. The structure of the_config parameter is similar to other implementations of AutoMoDe. For new modules the same structure is used. To get used to the structure I recommend using the AutoMoDe Editor [Kuckling et al., 2021b] as described in subsection 3.6 (AutoMoDe Editor Dr. Kuckling).

2.4 Reference Model

The reference model translates the ROS 2 publications of the robot to a AutoMoDe internal `robotState`. This is done to decouple hardware from decision logic. One could also think of it as an adapter. If the robot platform is changed, adaptions need to be done here to accept new or changed sensor information. If new sensor data is added, I strongly recommend adding it to the `robotState`, handling it in the reference model, and passing it to the modules in this way. While a direct subscription in the individual modules is possible with change in the behavior or condition node, it destroys the separation in the structure. In addition to that the reference model also calculates the `cmd_vel` for the robot, from the `wheels_speed` given by the behavior modules. Figure 1 gives a better intuition of the data flow. Different robot platform will use different reference models, but the basic structure should stay the same.

2.5 Behavior node, Condition node

The behavior and the condition ROS 2 nodes are in the package `automode_controller`. They execute and manage the behaviors and condi-

tions. These are dynamically loaded from a given package or the standard `basic_modules` package. They provide a list of available modules via a ROS 2 topics and services (ex. `behaviors/list` and `behaviors/list_srv`). They also provide the modules in form of ROS 2 actions to the controller. These actions are defined in the `automode_interface` package. In that role, the behavior node and the condition node are action servers and accept the request from the controller. One of the main differences of them is the ability of the condition node to host multiple conditions in parallel, where the behavior node host the actions mutually exclusive. On the other side they also host the loops for modules, sitting at 20Hz. For debugging the nodes log at the startup which modules are discovered properly. After that the log the current running modules. When implementing new modules I highly recommended to start the behavior or condition node separately and have a look at the logs as well as the outputs of the topic or service

2.6 Behavior and Condition Modules

The behavior and condition modules are a central feature of the AutoMoDe software. As they are the basis for all behavior that can be created and therefore also limit it, making them adjustable and alterable is key. To enable that and make it as easy as possible, they are separated in a different package. One could have multiple of these packages and switch between them, with giving the name of the package as ROS 2 argument to the behavior and condition node. If no argument is given, they use the package `basic_modules`. These packages need to consist of two folders (condition and behavior) with all modules in it. The two folders clearly separate modules for acting and sensing, later used as states and transitions in the finite state machine. The modules itself needs to match the given interface. They are all designed to run until canceled. This lets the controller decide which behavior or condition is active at a time and therefore ensures separation of concerns. If the conditions are satisfied (become true) they should give that back in the feedback but still not finish. The behaviors as well as the conditions get the necessary sensor information about the robot through the `robotState` topic. In addition, the behaviors publish on the topic `wheel_speeds` their actuator commands. More information about that are in the interfaces subsection 2.7 (Interfaces). Both module types work on a one step execution. The loop doing the steps is one level above in the nodes. These nodes also create the actions out of the modules. More information about how to set up new modules packages are found in the how to section 3 (How to do). To get started I suggest changing at first the existing example package. This consists of the modules listed in Table 2.

Module Name	Short Explanation
Behaviors	
behavior_interface	Abstract base class for all behavior modules; defines required interface for behaviors.
behavior_anti-phototaxis	Implements anti-phototaxis: robot moves away from detected light sources.
behavior_attraction	Implements attraction behavior: robot moves toward other robots.
behavior_exploration	Implements exploration: robot moves randomly and explores the environment.
behavior_phototaxis	Implements phototaxis: robot moves toward detected light sources.
behavior_repulsion	Implements repulsion: robot moves away from other robots.
behavior_stop	Implements stop behavior: robot halts all movement.
Conditions	
conditions_interface	Abstract base class for all condition modules; defines required interface for conditions.
condition_black_floor	Triggers when the robot detects a black floor surface.
condition_fixed_probability	Triggers randomly with a fixed probability each tick.
condition_inverted_neighbours_count	Triggers when neighbour count is below a threshold.
condition_neighbour_count	Triggers when neighbour count is above a threshold.
condition_white_floor	Triggers when the robot detects a white floor surface.
condition_floor_color	Triggers when the robot detects the floor color given in the argument

Table 2: Modules in the `basic_modules` package

2.7 Interfaces

In the interface package the action types for behaviors and conditions are described, as well as the message type for the `robotState`. These are given in the tables 3, 4 and 5. Its standard and necessary for ROS 2 python implementations to have these declarations in a separated package. Primarily to allow re-usability and prevent circular dependencies. The `robotState` is the internal representation of the sensor data, given by the robot. It is created from the reference model, accessed by all conditions and if necessary by the behaviors. I highly recommend to use this interface for all sensor data and don't subscribe directly in the modules to publications from the robot. On how to add sensor data correctly is described in the subsection 3.3 (Adding new sensor data).

Section	Field	Type
Goal	behavior_name params	string string[]
Result	success message	bool string
Feedback	(none)	

Table 3: Fields in Behavior.action

Section	Field	Type
Goal	condition_name params	string string[]
Result	success message	bool string
Feedback	condition_met current_status	bool string

Table 4: Fields in Condition.action

Field	Type	Description
robot_id	uint32	Unique identifier for the robot in the swarm
stamp	builtin_interfaces/Time	Timestamp when the message was generated
ground_black_floor	string	'gray', 'white' or 'black' as the floor is
proximity_magnitude	float64	Strength of the detected obstacle vector
proximity_angle	float64	Direction of the strongest proximity signal
light_magnitude	float64	Magnitude of detected ambient light
light_angle	float64	Direction of the light source
attraction_angle	float64	Direction toward neighbouring robots
neighbour_count	uint32	Number of neighbouring robots detected

Table 5: Fields in RobotState.msg

2.8 Development and Testing Tools

The `automode_tools` package provides a set of utilities to facilitate development, testing, and configuration of the AutoMoDe implementation. The tools allow developers to test new behavior and condition modules without the use of a simulator as well as creating configuration files for the AutoMoDe Editor [Kuckling et al., 2021a].

- `behavior_test_node` and `condition_test_node`: These node enable isolated testing of the behaviors and conditions. See also subsection 3.4 (Adding new modules)
- `robotState_test_pub`: This node simulates robot sensor without the use of Gazebo, providing a lightweight mechanism for testing module responses.
- `categories_creator`: A utility for reading available modules from a given module package via the behavior and condition nodes and generating finite state machine configuration files compatible with the AutoMoDe Editor [Kuckling et al., 2021b]. This allows easy editing and switching between different module packages. See also See also subsection 3.6 (AutoMoDe Editor Dr. Kuckling)

This structure ensures that module development, testing, and deployment can be done efficiently and systematically, reducing errors and improving maintainability. For example usage of these tools in combination with the launch files and missions, see section 3 (How to do).

2.9 Launch files

The implementation contains three main ROS 2 launch files.

`automode_tb4.launch.py` starts the condition, behavior, ref_model and controller node. It passes on the arguments `robot_namespace` as a namespace under which all nodes and topics are working, `module_package`, which defines the used module package and `fsm_config`. The `fsm_config` itself defines the finite state machine configuration. It also ensures that the controller waits for the behavior and condition node to be ready before startup.

`turtlebot4_simulation.launch.py` starts the simulation and the first robot. Additionally it starts the added virtual sensors for the TurtleBot 4. As arguments it takes `turtlebot4_id` as namespace for the robot and `world` as the map, which is started.

`add_turtlebot4_simulation.launch.py` adds any additional robot that is placed in the simulation as well as their virtual sensors. As arguments it takes `turtlebot4_id` as the namespace of the added robot and `world` which needs to match the world in previously launched `turtlebot4_simulation.launch.py`.

This separation is done to clearly separate simulation of environment and robots from controlling the robot as well as further use of the controlling part for real robots.

An overview is given in the table 6.

Launch File	**Argument**	**Default Value**

`automode_tb4.launch.py`	`robot_namespace`
	`module_package`
	`fsm_config`
		`tb1`
		`basic_modules`
		`-fsm-config`
		`-nstates 1 -s0`
		`0 -rwm0 50`

`turtlebot4_simulation.launch.py`	`turtlebot4_id` `world`	`tb1` `mission`
`add_turtlebot4_simulation.launch.py`	`turtlebot4_id` `world`	`tb2` `mission`

Table 6: ROS 2 launch files with their arguments and default values.

I want to point out that the files `light_sensor_node.py`, `ir_sensor_node.py` and `tf_broadcaster_launch.py` were provided by my supervisor and modified by me. At this point, the original author is unknown to me.

3 How to do

3.1 Install

In the following I describe how one can do a clean installation in a Virtual Machine running on a MacbookPro (arm). However the installation steps also work on a native installation of Ubuntu 24. If one has already a installation of ROS 2 jazzy, start with 3.1.4.

3.1.1 OS installation

For a Operating system I suggest using the arm version for Ubuntu 24, which can be downloaded direct from the ubuntu webpage:

`https://ubuntu.com/download/server/arm`

This can be used in a UTM with virtualization and works with reasonable performance. Once the image is started, one has to install the desktop environment.

`https://mac.getutm.app/`

Install desktop environment

```
sudo apt update
sudo apt install ubuntu-desktop
sudo reboot
```

As this setup always takes long for startup I suggest the following adaptation, which disables the wait for a network service to come up:

```
sudo systemctl disable systemd-networkd-wait-online.service
```

3.1.2 ROS 2 installation

Follow these instructions to install ROS 2. Make sure to install jazzy and source after installation.

3.1.3 Dev tools and simulator installation

Install the ROS 2 dev tools and the Gazebo simulator

Dev tools and simulator

```
sudo apt install ros-dev-tools
sudo apt-get install curl
sudo curl https://packages.osrfoundation.org/gazebo.gpg --output /
    usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
    keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.
    osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main
    " | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev
    /null
sudo apt-get update
sudo apt-get install gz-harmonic
```

3.1.4 Create ROS 2 workspace

Create the workspace with rosdep.

Workspace

```
mkdir ros2_ws && cd ros2_ws
sudo rosdep init
rosdep update
```

3.1.5 Create3 installation

Get the git repositories for the create3.

Create3 download

```
mkdir create3_ws && cd create3_ws
mkdir src && cd src
git clone https://github.com/paoloelle/create3_sim.git -b jazzy
git clone https://github.com/iRobotEducation/irobot_create_msgs -b
     jazzy
```

Before building, search for the file `create3.urdf.xacro` and change the line 288 to:
`<publish_link_pose> true </publish_link_pose>`

Now build and source it.

Create3 build

```
cd ..
rosdep install --from-path src -yi --rosdistro jazzy
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/local_setup.bash
cd ..
```

3.1.6 TurtleBot 4 installation

Download the repositories for the TurtleBot 4.

TurtleBot 4 download

```
mkdir turtlebot4_ws && cd turtlebot4_ws
mkdir src && cd src
git clone --branch jazzy https://github.com/turtlebot/turtlebot4.
    git
git clone --branch jazzy https://github.com/turtlebot/
    turtlebot4_setup.git
git clone --branch jazzy https://github.com/turtlebot/
    turtlebot4_desktop.git
git clone --branch jazzy https://github.com/turtlebot/
    turtlebot4_simulator.git
```

Make the following changes as the make the new worlds of the implementation available and improve the performance:
1. In `sim.launch.py` add the following below line 59:
`pkg_automode_tb4_sim = get_package_share_directory( 'automode_tb4_sim')`

2. Also in `sim.launch.py` add the following below line 68:
`os.path.join(pkg_automode_tb4_sim, 'worlds'),`

3. If the lidar is not used comment out or delete also line 37 to 50 in `rplidar.urdf.xacro` for better performance.

Now build and source it.

Turlebot 4 build

```
cd ..
rosdep install --from-path src -yi --rosdistro jazzy
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/local_setup.bash
cd ..
```

3.1.7 AutoMoDe installation

Now download and build AutoMoDe

AutoMoDe install

```
git clone https://github.com/benkroe/automodeROS.git
cd automodeROS
colcon build --symlink-install
source install/setup.bash
```

The code is also available at:

AutoMoDe install

```
git clone https://github.com/StudentWorkCPS/automode-ros.git
```

3.1.8 Optional: Add sources to `.bashrc`

If the implementation is used I recommend adding the source commands to the `.bashrc` file to ensure that installations are always sourced if new terminals are used. To do that add at at the end of your `.bashrc` file the following lines. To open `.bashrc` use `vim ~/.bashrc` or preferred tools and add these lines at the end. Adapt them if the installation takes place somewhere else then home.

AutoMoDe install

```
# Source ROS 2 and workspace setup files
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/create3_ws/install/setup.bash
source ~/ros2_ws/turtlebot4_ws/install/setup.bash
source ~/ros2_ws/automodeROS/install/setup.bash
```

3.2 Start the example missions

I provide two example missions with two TurtleBot 4 each. Before starting the examples as shown below, ensure all necessary packages are sourced correctly. The missions are primarily for showcasing the capabilities of the implementation as well as a entry point for getting used to it.

3.2.1 Start the robots tb1 and tb2

To start the TurtleBot 4s run the two launch files in separated terminals and wait after the first launch proper time, especially on resource restricted systems. `add_turtlebot4_simulation.launch.py` should also work to add more robots if enough computational power is available. In the VM I wasn't able to run a third or more robots.

Start tb1 and tb2

```
ros2 launch automode_tb4_sim turtlebot4_simulation.launch.py
    turtlebot4_id:=1 world:=mission
ros2 launch automode_tb4_sim add_turtlebot4_simulation.launch.py
    turtlebot4_id:=tb2 world:=mission
```

In Figure 3 the TurtleBots can be seen in the Gazebo simulation. On the left side the yellow ball represents the light source in the environment. The space for the robots is surrounded by octagonal brown walls, it has 3 floor colors: white, gray and black which can be detected by the robots. On the right side the controls from the `turtlebot4` package are present. Videos of the missions are available at mission videos [4]

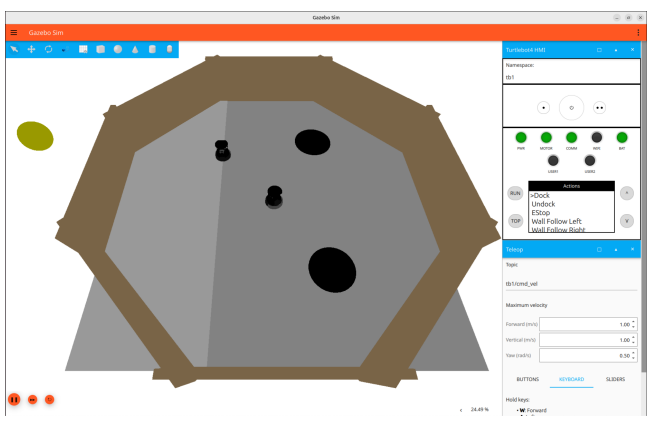

Figure 3: Two TurtleBot 4 in a Gazebo simulation performing the mission

[4] TurtleBot 4 mission videos: `https://1drv.ms/f/c/689078a1e7ff6d99/EnyVsOh5tUNCou-aOUmpj04B6mhF_PEni1ziHT3qqC40Bg?e=gYSbNZ`

3.2.2 Mission 1: Foraging

In foraging both robots are started with the same finite state configuration. The task is to collect coin by driving over the black patch and deliver them to the white patch This is done with the help of the light source as it directs the way to the white patch and give and also a direction where to find the black ones. The finite state machine for that is given in Figure 4.

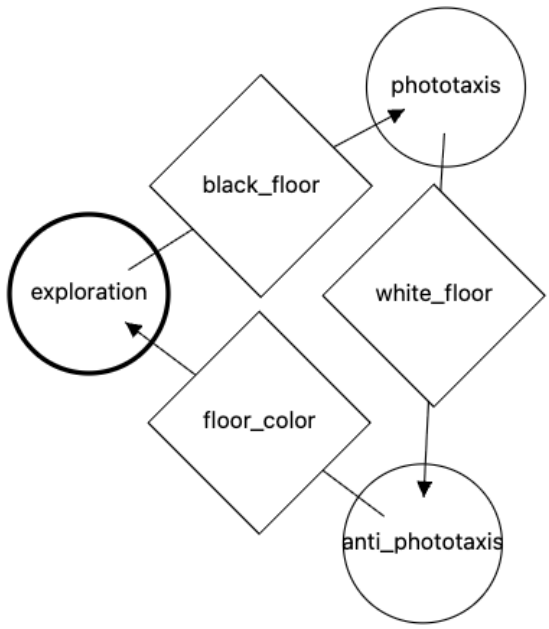

Figure 4: –fsm-config –nstates 3 –s0 0 –rwm0 50 –n0 1 –n0x0 0 –c0x0 2 –p0x0 1 –s1 2 –n1 1 –n1x0 1 –c1x0 1 –p1x0 1 –s2 3 –n2 1 –n2x0 0 –c2x0 0 –color2x0 2 –p2x0 1

After launching two robots in the simulation start the controller for them. The second TurtleBot 4 is started by the same command but changing the namespace to `robot_namespace:=tb2`

Start controller

```
ros2 launch automode_controller automode_tb4.launch.py fsm_config
    :="--fsm-config --nstates 3 --s0 0 --rwm0 50 --n0 1 --n0x0 0
    --c0x0 2 --p0x0 1 --s1 2 --n1 1 --n1x0 1 --c1x0 1 --p1x0 1 --
    s2 3 --n2 1 --n2x0 0 --c2x0 0 --color2x0 2 --p2x0 1 "
    robot_namespace:=tb1
```

3.2.3 Mission 2: Find and catch

In find and catch the first TurtleBot 4 executes a simple one state finite state machine with `exploration`. The second TurtleBot 4 explores till it finds the first one. Then it tries to follow. When getting lost, it explores again. Figure 5 displays the finite state machine for the second robot.

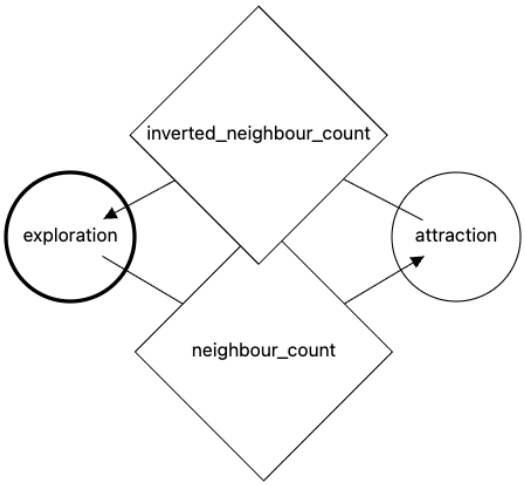

Figure 5: –fsm-config –nstates 2 –s0 0 –rwm0 30 –n0 1 –n0x0 0 –c0x0 3 –p0x0 1 –w0x0 1 –s1 4 –att1 1 –n1 1 –n1x0 0 –c1x0 4 –p1x0 1 –w1x0 1

After launching two robots in the simulation start the controller for them. Do this in separated terminals.

Start controller

```
ros2 launch automode_controller automode_tb4.launch.py fsm_config
    :="--fsm-config --nstates 1 --s0 0 --rwm0 30"
ros2 launch automode_controller automode_tb4.launch.py fsm_config
    :="--fsm-config --nstates 2 --s0 0 --rwm0 30 --n0 1 --n0x0 0
    --c0x0 3 --p0x0 1 --w0x0 1 --s1 4 --att1 1 --n1 1 --n1x0 0 --
    c1x0 4 --p1x0 1 --w1x0 1" robot_namespace:=tb1
```

3.2.4 Mission Conclusion

Both example missions can be started consistently, which indicates a robust system. Although a lot of warnings are created from ROS 2, Gazebo and also the TurtleBot 4 package, which indicates quite the opposite. In the simulation the TurtleBots performed as intended with the given finite state machine con-

figurations. The performance of the system for both missions is very low with working generally at 15% to 20% of real time. Brief analyses of CPU usages have shown that most computational power is used for Gazebo with the TurtleBot 4 model and very few for the controller part. It is clear that the use of a VM creates significant overhead and disables the direct use of GPU power, as it is all emulated. But the creation process of the missions showed also the benefits of the highly adaptable system as making changes and corrections was easy and fast. The extensive logging of errors helped with that. I am personaly interested and motivated to create new wordls and task for the TurtleBots in this environment.

3.3 Adding new sensor data

To add new sensor data one needs to ensure that the data is available via a ROS 2 topic. I describe the procedure with the example of a `random_sensor`, which gives just a random number. The changes are also presented in the code as comments. At first the `ref_model_turtlebot4_gz.py` needs to subscribe to the given ROS 2 topic. Therefore we add the subscription in the `__init__` method.

Subscribe to random_number

```
self.create_subscription(String, 'random_number', self.
    _random_number_cb, qos_profile_sensor_data)
```

In the `__init__` we also add the new variable to the example state and add a variable for the latest stored value.

Add necessary variables

```
self.random_number = 4
self.latest_random_number = 4
```

As we created the description to a callback function, we also need to add a callback function.

Create callback method

```
def _random_number_cb(self, msg):
    self.latest_random_number = msg.data
```

To get the sensor data in the behavior and condition modules, we need to add it to the `robotState`. For this add it in the `_publish_robot_state` method.

Add random number to robot_State

```
msg.random_number = self.latest_random_number
```

After that we can use the new field in `robotState` in the modules. For example in the `behavior_stop.py` we add in the `execute_step`

Log the random number

```
self._node.get_logger().debug(f'Random number: {self.
    _last_robot_state.random_number}')
```

Note that the behaviors need to subscribe to the `robotState`. Also, if necessary, the modules can subscribe sensor data directly, but I strongly recommend going over the `ref_model_turtlebot4_gz.py` node.

3.4 Adding new modules

To add new modules one could either add the modules to the existing `basic_modules` package or create a new package. If a new package is created, ensure the structure is correct. In the package a folder is created, the same name as the package. In that the folders behaviors and conditions. In them, the modules. To use the package with the `automode_tb4.launch.py` launch file, use the parameter `module_package:=<new_package_name>`.

New modules need to inherit from the `BehaviorBase` or `ConditionBase` class. The `BehaviorBase` class if provided below to clarify the following explanations. I recommend to copy them to every new modul package. On runtime these classes give a warning, because the nodes want to load them. This is intended and gives the hint that the whole folder is searched and the import works. As an example the `BehaviorBase` is given below. Special attention needs to be on the `get_description` method, as this description is later used by the controller to create the finite state machine. Make sure to match the structure of base classes or given modules. In the `setup_communication` method the modules can subscribe to the `robotState` and/or `wheel_speed` over the parent node. I recommend not subscribing to anything else as it would destroy the intended structure. In the `execute_step` method one step of the behavior or condition is executed. Do not loop here, as the loop is done in the parent node.

BehaviorBase class

```
#!/usr/bin/env python3

from abc import ABC, abstractmethod
from typing import Dict, Any, Tuple

class BehaviorBase(ABC):
    @staticmethod
    @abstractmethod
    def get_description() -> Dict[str, Any]:
        """Return metadata: name, params, description"""
        """It needs:
          name: String
          type: String --> is used later in the config
          description: String
          params: [param: {name: String, type: Any, required:
    Boolean, default: Any}]
          """

    @abstractmethod
    def setup_communication(self, node) -> None:
        """Attach pubs/subs to provided rclpy.Node"""

    @abstractmethod
    def set_params(self, params: Dict[str, Any]) -> None:
        """Configure behavior instance"""

    @abstractmethod
    def execute_step(self) -> Tuple[bool, str]:
        """Publish/act for one controller tick; return (success,
    message)"""

    def reset(self) -> None:
        """Optional: tear down publishers/subscriptions"""
        return
```

To test new modules without the need of a simulation, the nodes `behavior_test_node`, `condition_test_node` and `robotState_test_node` can be used together with the `behavior_node` and `condition_node`. Therefore the nodes need to be started in separated terminals. The `basic_modules` can be replaced with the new package name. If new sensor data is used the `robotState_test_node` needs adaptation to publish new demo data. After changing the modules a clear rebuild is sometimes needed (delete build, install, log before building).

Test behavior modules

```
ros2 run automode_controller behavior_node module_package:=
    basic_modules
ros2 run automode_tools behavior_test_node
ros2 run automode_tools robotState_test_pub
```

3.5 Add new worlds

New worlds (simulation environments) are created in the `automode_tb4_sim` package, worlds folder. Two worlds are already implemented: `white.sdf` is a big sqared world with two non-static cylinders, for obstacle detection, a light in the middle, a white patch in the middle and a black patch in one corner. `mission.sdf` is a smaller octagonal world with two black patches, one white patch with a light above it. It is quite similar to other worlds used in previous AutoMoDe research by Ligot et al. [2020]. For resource restricted setups, use the smaller worlds with few robots. Otherwise, feel free to use the big one with a lot of robots. To create a new world, place the files next to the given files and add them in `setup.py` to make it available. If worlds from another package are used, the package needs to be added in `sim.launch.py` of the `turtlebot4` download and the TurtleBot 4 needs to be build again. See the install section, where the worlds of `automode_tb4_sim` are added.

3.6 AutoMoDe Editor Dr. Kuckling

Kuckling et al. [2021a] introduce a editor for finite state machines. In `automode_tb4_sim` I created a wrapper node that creates the necessary config files from a modules package. To use it run the `behavior_node` and `condition_node` from the `automode_controller` package, with the module package as argument. Then run the `categories_creator` from the `automode_tools` package. This creates two config files. Replace the existing config files in the editor. How this is done is explained in the techreport for the AutoMoDe Editor [Kuckling et al., 2021b]. After that, the editor is working with the used modules package and finite state machines can be created or looked at in a natural form.

Create categories_config files

```
ros2 run automode_controller behavior_node modules_package:=
    basic_modules
ros2 run automode_controller condition_node modules_package:=
    basic_modules
ros2 run automode_tools categories_creator
```

Please be aware that the wrapper cannot know the minimum and maximum values for the modules, as they are not defined in them, so it chooses the appropriate ones based on the variable type. This results, for example, in the editor allowing values greater than 1 for probabilities. Eater be aware of that or modify the generated categories files.

4 Discussion

Compared to other implementations of AutoMoDe, this implementation improves on adaptability, usability and separation of individual components. The use of ROS 2 as a middleware enabled a solid separation of concerns and enabled the introduction of the TurtleBot 4 to the AutoMoDe robot platform family. Using Gazebo also brought a new simulator to the field. But it also under-performs because large swarms and offline training seem not possible. This shows why the ARGoS3 simulator is so popular in the publications around AutoMoDe as it clearly outperforms Gazebo with large swarms and running multiple simulations in parallel while keeping speeds, which far exceed real time. Further assessment needs to be done, to show if a native Ubuntu installation with real GPU support could reduce the performance gap to other implementations of AutoMoDe or if the overhead generated by the middleware ROS 2 is too heavy, even with other simulators. In my opinion, this project provides benefit as it tries something new in a field that has a long history with many publications. It points out a new direction and idea to move forward.

References

Mauro Birattari, Antoine Ligot, and Gianpiero Francesca. *AutoMoDe: A Modular Approach to the Automatic Off-Line Design and Fine-Tuning of Control Software for Robot Swarms*, pages 73–90. Springer International Publishing, Cham, 2021. ISBN 978-3-030-72069-8. doi: 10.1007/978-3-030-72069-8_5. URL `https://doi.org/10.1007/978-3-030-72069-8_5`.

Gianpiero Francesca, Manuele Brambilla, Arne Brutschy, Vito Trianni, and Mauro Birattari. Automode: A novel approach to the automatic design of control software for robot swarms. *Swarm Intelligence*, 8(2):89–112, 2014. doi: 10.1007/s11721-014-0092-4. URL `https://doi.org/10.1007/s11721-014-0092-4`.

Miquel Kegeleirs, R Todesco, D Garzón Ramos, G Legarda Herranz, and Mauro Birattari. Mercator: hardware and software architecture for experiments in swarm slam. *IRIDIA, ULB, Tech. Rep. TR/IRIDIA/2022-012*, 2022.

N. Koenig and A. Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, volume 3, pages 2149–2154 vol.3, 2004. doi: 10.1109/IROS.2004.1389727.

J. Kuckling, K. Hasselmann, V. van Pelt, C. Kiere, and M. Birattari. Automode editor: a visualization tool for automode. Technical Report TR/IRIDIA/2021-009, IRIDIA, Université libre de Bruxelles, Brussels, Belgium, 2021a.

Jonas Kuckling, Ken Hasselmann, Vincent Van Pelt, Cédric Kiere, and Mauro Birattari. Automode editor: a visualization tool for automode. Technical report, Tech. Rep. TR/IRIDIA/2021-009 (IRIDIA, Université libre de Bruxelles . . . , 2021b.

A. Ligot, J. Kuckling, D. Bozhinoski, and M. Birattari. Automatic modular design of robot swarms using behavior trees as a control architecture. *PeerJ Computer Science*, 6:e314, 2020. doi: 10.7717/peerj-cs.314. URL `https://dx.doi.org/10.7717/peerj-cs.314`.

Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66): eabm6074, 2022. doi: 10.1126/scirobotics.abm6074. URL `https://www.science.org/doi/abs/10.1126/scirobotics.abm6074`.

Olivier Michel. Cyberbotics ltd. webots™: professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):5, 2004.

Francesco Mondada, Michael Bonani, Xavier Raemy, James Pugh, Christopher Cianci, Adam Klaptocz, Stephane Magnenat, Jean-Christophe Zufferey, Dario Floreano, Alcherio Martinoli, et al. The e-puck, a robot designed for education in engineering. In *Proceedings of the 9th conference on autonomous robot systems and competitions*, volume 1, pages 59–65. Castelo Branco: IPCB, Instituto Politécnico de Castelo Branco, 2009.

Carlo Pinciroli, Vito Trianni, Rehan O'Grady, Giovanni Pini, Arne Brutschy, Manuele Brambilla, Nithin Mathews, Eliseo Ferrante, Gianni Di Caro, Frederick Ducatelle, Mauro Birattari, Luca Maria Gambardella, and Marco Dorigo. Argos: a modular, parallel, multi-engine simulator for multi-robot systems. *Swarm Intelligence*, 6(4):271–295, 2012. doi: 10.1007/s11721-012-0072-5. URL `https://doi.org/10.1007/s11721-012-0072-5`.